SECURITY WHITEPAPER

Customer-Held Controls for Data Sovereignty in Snowflake

Architecture Patterns Where the Customer Holds the Key, the Data, or the Switch

Kevin Keller • Principal Security Architect & Field CTO, Snowflake

Kevin Keller, Principal Security Architect & Field CTO at Snowflake, writing in a personal capacity

> **Personal publication.** This is the author's own white paper, not a Snowflake white paper. It is not reviewed,
> approved or endorsed by Snowflake. The patterns and code are "art of the possible" examples for inspiration, not
> official Snowflake solutions, products or recommendations, and they come without support or warranty.

Contents

1. Document Purpose and Disclaimers
2. Introduction
3. Threat Model
4. Baseline: Native Snowflake Controls
5. Pattern 1 – On-Demand Tunnel to an On-Premises Lake
6. Pattern 2 – Column Encryption with an External Key Broker and Step-Up
7. Pattern 2b – Key Fetch from an External Access UDF
8. Pattern 2c – Device-Code Approval and Query-Bound Key Release
    - Binding a Key Request to the Running Query
9. Pattern 2d – Remote Decryption: the Key Never Enters Snowflake
10. Pattern 3 – Split Storage with a Local Join
11. Pattern 4 – Ciphertext in Snowflake, Decryption in an On-Premises Application
12. Complement – Homomorphic Encryption for Numeric Columns
13. Comparing and Combining the Patterns
    - Relation to the EU Cloud Sovereignty Framework (SEAL)
14. Security Caveats and Operational Considerations
15. Reference Implementation
16. Conclusion

Document Purpose and Disclaimers

Organizations that put their analytics on a cloud data platform sometimes hold a small set of data whose control requirements go further than what a shared platform provides by default. Examples include national identifiers, payroll, health records, and data covered by sector or national regulation. For these datasets, the question is not only is it encrypted? but **who must take part before anyone, including the platform operator, can read it?**

This whitepaper describes architecture patterns in which **the customer holds the decisive control**: the key, the data itself, or the switch that makes the data reachable. Snowflake remains the analytics platform for the rest of the estate. Each pattern places the trust boundary in a different spot and comes with a working reference implementation and measured behavior.

This document is intended for:

- **Security and enterprise architects** who evaluate how much control over sensitive data they can retain while using Snowflake.
- **Data protection officers, compliance and risk teams** who need concrete documentation of where data lives, who holds keys, and what is audited.
- **Platform engineering teams** who would build and operate one of these patterns.

The patterns are **custom architectures**, not Snowflake product features. They combine documented Snowflake capabilities (Snowpark Container Services, external functions, External Access Integrations, masking policies, `DECRYPT_RAW`, query history) with components that the customer operates. Some behavior described here is observed rather than documented; such behavior is marked as such and should be treated as subject to change.

The descriptions reflect Snowflake's platform as of late 2026. Statements about features, and any forward-looking statements, are subject to change. This document makes no claim to be complete or current. Verify features against the current Snowflake documentation before making architectural decisions.

The views expressed here are the author's own and do not necessarily reflect those of Snowflake. They are provided for informational purposes. This document does not constitute legal, compliance, or contractual advice, and it makes no statement about how any specific law or regulation applies to a given organization.

Introduction

Snowflake already encrypts all data at rest and in transit, and it offers customer-managed keys through Tri-Secret Secure, private connectivity, and fine-grained governance (see Baseline). For most data this is the appropriate level of control.

A minority of datasets raise a stricter question: can the data be read without the data owner's active participation? For example:

- a request served on the platform operator,
- an administrator with broad privileges in the account,
- stolen credentials.

The patterns in this paper answer that question in different ways:

1. **Pattern 1 – On-demand tunnel.** The sensitive data stays in an on-premises Apache Iceberg lake. Snowflake can query it only while the data owner has opened an outbound tunnel, for a time-boxed window, and results are never cached.
2. **Pattern 2 – External key broker with step-up.** Sensitive columns are stored in Snowflake as AES-256-GCM ciphertext. The key is released per query by a broker the customer runs, only after a person completes MFA with a written justification and only while that exact query is running.
3. **Pattern 2b – External Access key fetch.** A lighter variant of Pattern 2. A Python UDF fetches the key directly from the customer's vault and keeps it in memory for the duration of one query.
4. **Pattern 2c – Device-code approval and query-bound key release.** Pattern 2b combined with Pattern 2's controls: an OAuth device-code approval on a second device yields an API key scoped to one user, account and session. Every key release is bound to the query's own running query ID.
5. **Pattern 3 – Split storage with a local join.** Only non-sensitive attributes are stored in Snowflake. Sensitive attributes never leave the premises, and joins happen in local memory.
6. **Pattern 4 – Client-side decryption.** Ciphertext is stored in Snowflake, but the key exists only in an on-premises application. Snowflake never sees a key, a plaintext value or a plaintext result.
7. **Complement – Homomorphic encryption.** Snowflake computes sums, averages and model scores on encrypted numbers. Only the data owner can decrypt the results.

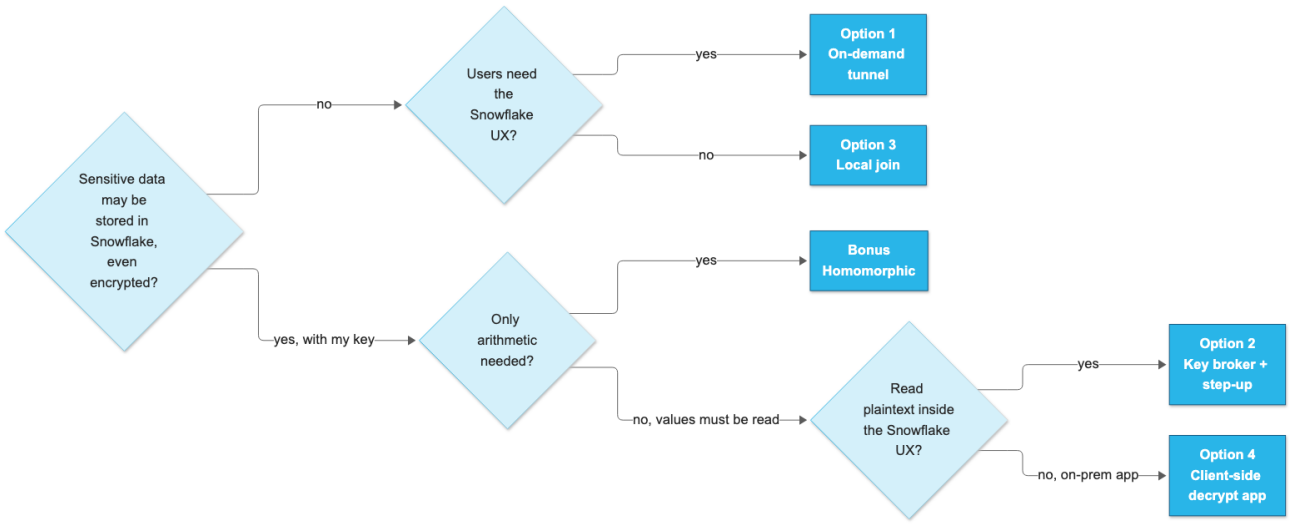


Figure 1 – A first orientation: may the data be stored in Snowflake at all, must users read individual values, and must they do so inside the Snowflake user experience? The patterns can be combined.

Threat Model

The patterns are designed against the following scenarios. Neutral wording is deliberate: the same controls address a curious operator, a compromised credential and a compelled disclosure.

#	Scenario	Description
T1	**Data at rest read by the platform operator**	A party with access to the platform's storage or infrastructure attempts to read stored data.
T2	**Compelled disclosure through the platform**	A third party, including a public authority, requires the platform operator to produce customer data. The aim of the patterns is that such a request can only be fulfilled with the customer's participation.
T3	**Over-privileged account administrator**	A user with broad roles (for example `ACCOUNTADMIN`) queries sensitive data outside their duty.
T4	**Stolen Snowflake credentials**	An attacker authenticates as a legitimate user.
T5	**Replay of an earlier authorization**	A key, token or approval captured once is reused later or from elsewhere.
T6	**Authorized user exfiltrates what they may see**	A legitimate user copies results they are allowed to read.

Table 1 — Scenarios each pattern is designed to resist

Scenario	P1 Tunnel	P2 Broker	P2b EAI	P2c Device flow	P3 Split	P4 Client decrypt	HE
T1 Data at rest	✓ not stored	✓ ciphertext	✓ ciphertext	✓ ciphertext	✓ not stored	✓ ciphertext	✓ ciphertext
T2 Compelled disclosure	✓ tunnel closed	✓ key held by customer	◐ key held by customer, released to the integration	✓ key held by customer, released per approved session	✓	✓	✓
T3 Over-privileged admin	✓ when closed	✓ no step-up, no key	◐ RBAC only	✓ no approval, no key	✓	✓	✓
T4 Stolen credentials	✓ when closed	✓ no TOTP, no grant	✗	✓ no TOTP, no API key	✓	✓ app login needed	✓
T5 Replay	n/a	✓ query must be running	✗	✓ session-scoped key and running query ID	n/a	n/a	n/a
T6 Authorized exfiltration	✗	✗	✗	✗	✗	✗	◐ aggregates only

✓ designed to resist · ◐ partly · ✗ not addressed

No pattern stops an authorized person from copying what they are allowed to see. That remains a matter of governance (no `CREATE TABLE` in decrypting roles, unload prevention, export controls) and of minimizing who receives such authorization.

None of the patterns protects plaintext while it is processed, except Patterns 3 and 4 (processing happens on the customer's side) and homomorphic encryption (processing happens on ciphertext). In Patterns 1 and 2–2c, plaintext and, for 2–2c, the data key exist in Snowflake compute memory for the duration of an authorized query. These patterns control **when** and **for whom** that happens; they do not make computation blind.

Baseline: Native Snowflake Controls

The patterns build on, and do not replace, the platform's native controls. Several of them already address parts of the threat model:

Capability	What it provides
Encryption at rest and in transit, hierarchical key model, periodic rekeying	All data encrypted by default
Tri-Secret Secure (Business Critical)	A composite master key built from a Snowflake-maintained key and a customer-managed key in the cloud provider's KMS; revoking the customer key makes the account's data unreadable
Private connectivity (AWS PrivateLink, Azure Private Link, Google Private Service Connect), network policies and rules	Control over network paths into and out of Snowflake
Dynamic data masking, row access policies, projection and aggregation policies	Column- and row-level control at query time
External tokenization	Masking policies that detokenize through an external function, so that Snowflake stores only tokens
`ACCESS_HISTORY`, `QUERY_HISTORY`, event tables	Audit of what was accessed, by whom

Tri-Secret Secure gives the customer an account-wide kill switch. The patterns in this paper add **finer granularity**: per column, per classification, per person, per session and per query. They also address the stricter requirement that some data is not stored in the cloud at all.

Pattern 1 – On-Demand Tunnel to an On-Premises Lake

Idea. Snowflake stays the strategic platform and holds most of the data. Ultra-sensitive tables stay on-premises in an Apache Iceberg lake. They are reachable from Snowflake **only while the data owner opens an outbound tunnel**, for a bounded time, with a recorded reason.

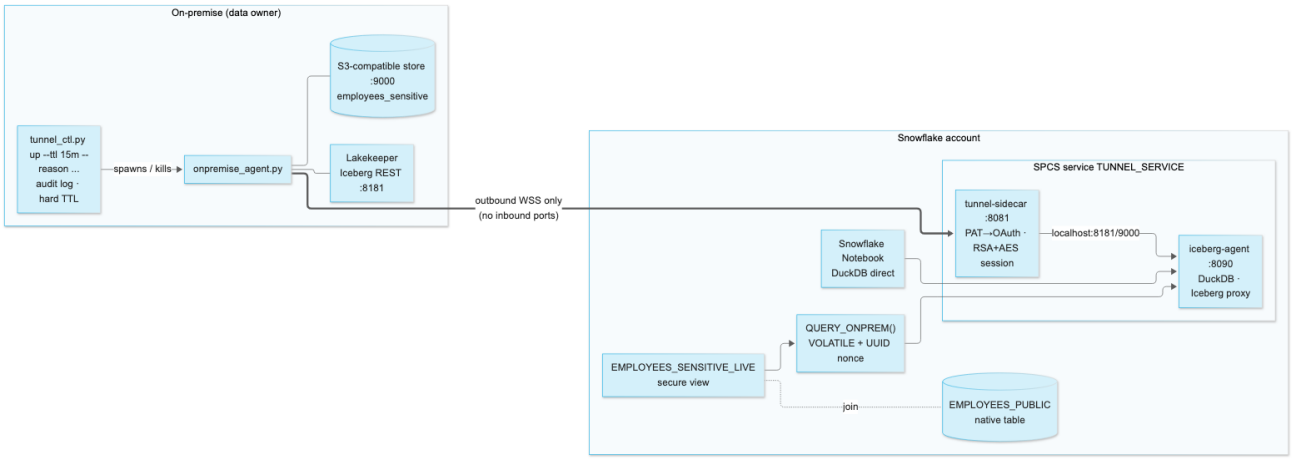


Figure 2 – An on-premises agent opens an outbound WebSocket connection to a Snowpark Container Services (SPCS) service. No inbound ports are opened. A `VOLATILE` SQL function and a secure view expose the remote table to Snowflake users.

How it works

1. The data owner runs `tunnel-up` with a time-to-live and a reason. The control script starts the agent and **terminates it at the TTL**, regardless of what else happens. Every open and close goes to an audit log.
2. If needed, the control script also resumes the compute pool and service. A cold start took about one minute in the reference environment.
3. The agent authenticates as a dedicated `TYPE=SERVICE` user with a role-restricted programmatic access token (PAT), limited by a network policy to the on-premises egress IP. It connects to the SPCS sidecar, which proxies the Iceberg REST catalog and object store to an in-service query agent.
4. Analysts query a secure view, `EMPLOYEES_SENSITIVE_LIVE`. It calls `QUERY_ONPREM()`, a `VOLATILE` wrapper that adds `UUID_STRING()` to every call, so the **result cache is never used**. They can join it with native Snowflake tables.
5. When the tunnel is closed, the same query returns no data. The view is empty and the function returns the agent's connection error as a value rather than raising a SQL error. There is no copy of the data to fall back on. Applications that must distinguish "no rows" from "tunnel closed" should check for that error value.

Measured behavior

- With the tunnel open, the join of the native `EMPLOYEES_PUBLIC` table and the live on-premises view returned the expected per-department averages.
- Four identical runs, two with `USE_CACHED_RESULT=TRUE` explicitly set: all four query IDs appeared in the on-premises agent log, so each execution went through the tunnel.
- After `tunnel-down`, no data came back: `QUERY_ONPREM` returned `Could not connect to server … localhost:8181` as its value, and the view returned 0 rows.
- Latency (X-Small warehouse, 200 rows): join and aggregate through the tunnel about 0.3 s, a single-row lookup about 0.24 s, against about 0.1 s for the same aggregate on plaintext in Snowflake. Opening the tunnel took about 14 s to the first answer with the service already running.

Considerations

Strengths	Limitations
Data is not stored in Snowflake at all	Data in transit and in query memory while the tunnel is open
The data owner holds a physical on/off switch with a TTL	Throughput is network-bound; suited to bounded queries
Snowflake user experience is kept: SQL, notebooks, joins with native data	Snowflake governance applies to the view and function, not to the remote rows
No inbound connectivity to the premises	SPCS service, on-premises agent and lake to operate

Variant: a notebook and a customer-chosen query engine

A notebook running on Snowpark Container Services can query through the tunnel with an engine the customer
chooses, such as DuckDB. This can be a Snowflake Notebook or a custom notebook such as Jupyter or marimo. No SQL
then runs on Snowflake's query engine (the virtual warehouse) for the sensitive data. No query result exists, so
nothing reaches the result cache or `RESULT_SCAN`, and the data exists only in the notebook container's memory for

the session. With a Snowflake Notebook, users keep the Snowflake experience: the editor, sign-in, RBAC, and joins
with Snowflake data in the same notebook.

The trade-off is the compute power and on-demand elasticity of Snowflake's query engine. The work is bounded by a
single notebook container, and Snowflake governance does not apply to that data path.

The reference implementation does not pass the notebook user's identity through to the on-premises source. The
agent authenticates to the Iceberg catalog with its own credentials. A production design would need one of these:

- access control handled out of band;
- a token from an external identity provider, for example through a device-code flow started from the notebook
  (as in Pattern 2c);
- an engineered proof of the Snowflake identity, taken for example from the user context, request headers or
  notebook secrets.

This is not covered here, because on-premises sources and their authentication requirements vary widely. An
example pattern for seamless Iceberg catalog authentication from a notebook on Snowpark Container Services is
planned.

Pattern 2 – Column Encryption with an External Key Broker and Step-Up

Idea. Sensitive columns are encrypted with AES-256-GCM before they reach Snowflake, with additional authenticated data (AAD) per classification. Snowflake holds ciphertext only. A masking policy asks an **external key broker**, operated by the customer, for the data key whenever a query needs it. The broker releases the key only if all of the following hold:

1. the request is signed by the Snowflake API integration's IAM role (API Gateway with IAM authorization and a resource policy),
2. the user completed a **step-up outside Snowflake** (TOTP and a written justification) that is still valid,

3. a **call-back into Snowflake** confirms that the exact `query_id` from the request header is `RUNNING` right now, issued by this user, under this role, in this account.

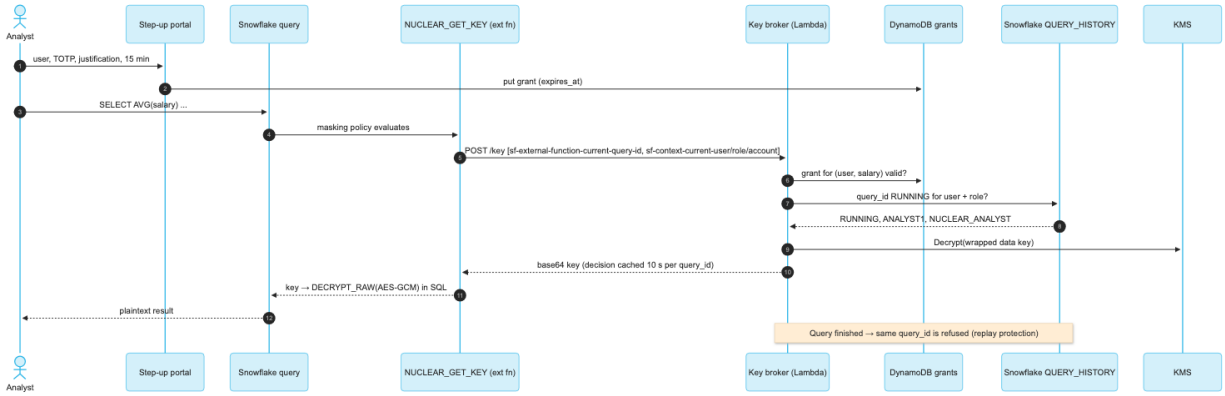


Figure 3 – The external function sends the query ID and context headers. The broker checks the step-up grant, calls back into Snowflake's query history, unwraps the data key with KMS and returns it. Decryption happens in SQL with `DECRYPT_RAW`.

Why an external function

External functions receive the current query ID and, through `CONTEXT_HEADERS`, the current user, role and account, **injected by Snowflake rather than passed as arguments**. The caller cannot forge them in SQL. Declaring the function `IMMUTABLE` with a constant argument lets Snowflake treat the key as a per-query constant. In addition, the broker caches allow decisions per query ID for 10 seconds and deny decisions for 60 seconds.

Measured behavior

Step	Result	Broker reason
Viewer role	`**ENCRYPTED**`, the policy never calls the broker	–
Analyst role without step-up	`**KEY DENIED**`	`no_stepup_grant`, no call-back needed
Analyst after TOTP and justification	Plaintext; aggregates computed in Snowflake	`running`, justification audited
Replay of that query ID after completion	`NULL` key	`query_not_running(SUCCESS)`
Grant revoked	`**KEY DENIED**`	`no_stepup_grant`

- **Call-backs:** one query decrypting five columns across 200 rows caused **two broker invocations**, one per classification, and two call-backs.
- **Latency:** a decrypting query over 200 rows took about 4 seconds (plaintext: 0.1 s), mostly the external function round trip. Limiting the query-history lookup to a recent time window was essential: with only a row limit it took 8–10 seconds for a busy user.
- **Result cache:** queries that contain external functions are not reused from the persisted result cache, according to Snowflake documentation, so every decrypting query is re-authorized.
- **Least privilege:** with only `MONITOR` on the warehouse, the broker's service user can see other users' running queries. As a side effect, decryption is limited to monitored warehouses.

Considerations

Strengths	Limitations
Full Snowflake user experience; decryption is transparent through masking policies	Key and plaintext exist in compute memory during an authorized query
Per-person, per-classification, time-boxed approval with justification	AWS stack to operate: KMS, Lambda, API Gateway, DynamoDB, API integration
Each release bound to an exact running query, so no replay	About 4 s for each decrypting query in the reference setup
Customer holds the key and the kill switch	Encrypted columns cannot be filtered or joined without decryption

Pattern 2b – Key Fetch from an External Access UDF

Idea. The same ciphertext model as Pattern 2, but without API Gateway and an API integration. A Python UDF with an **External Access Integration (EAI)** fetches the key from the customer's vault over HTTPS (optionally through outbound private connectivity) and keeps it in a module-level memo.

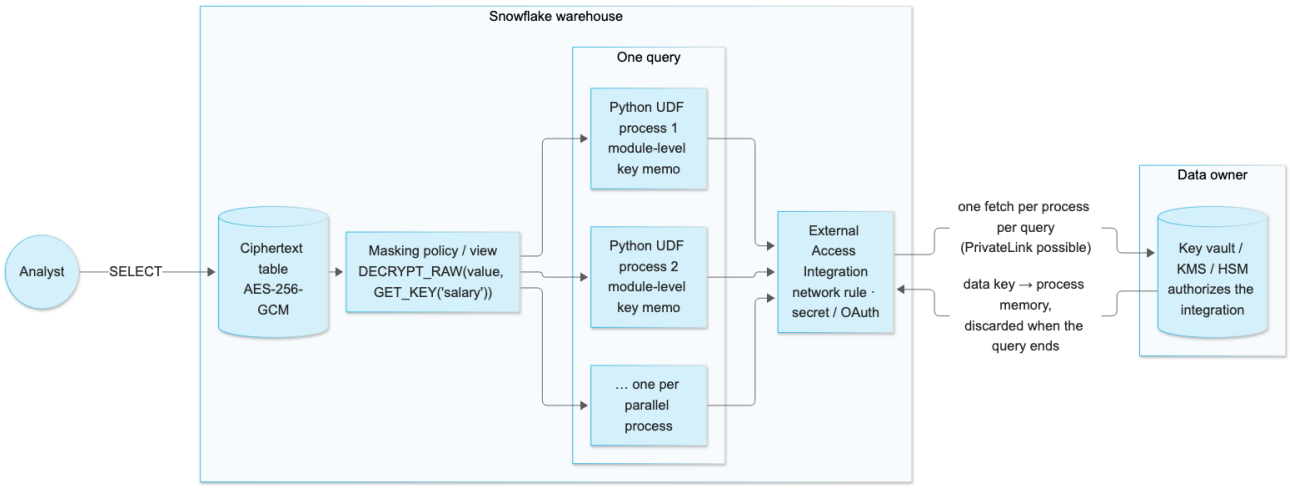


Figure 4 – Each parallel Python process of a query fetches the key once and keeps it in memory for the life of that process.

Measured memo lifetime

The claim that a UDF keeps a key cached "as long as the warehouse is warm" was tested and **not reproduced**:

Observation (X-Small warehouse, with and without EAI)	Result
Next query in a new session, under another role, or in the same session after 20 seconds	New Python process; memo empty
One query over 5,000 rows	8 parallel processes, 4,992 memo hits, so **8 key fetches instead of 5,000**

The memo therefore behaves as a **per-query cache**. This is observed behavior, not a documented contract. A design should be safe either way: give the memo a TTL, key it by user and role, and use short-lived tokens.

Considerations

Strengths	Limitations
Far less infrastructure: network rule, secret or OAuth, EAI	The vault authorizes the **integration**, not the person or the query
Works with any HTTPS key service, including through private connectivity	No step-up or replay protection out of the box
Key only in UDF memory for one query (as measured)	Authorization rests on Snowflake RBAC on the UDF and policy
Kill switch: disable the integration or revoke the token	Process lifetime is platform behavior, not a contract

If the vault should know who is asking and for which query, the customer adds a middleware in front of the vault. Pattern 2c is a worked example of that.

Pattern 2c – Device-Code Approval and Query-Bound Key Release

Idea. Pattern 2b's transport combined with Pattern 2's controls. A person approves access on a second device with MFA and a justification, using the OAuth 2.0 device authorization grant (RFC 8628). They receive an API key that is valid **only for their user, this account and this session**. Every key fetch is checked against the query's own running query ID.

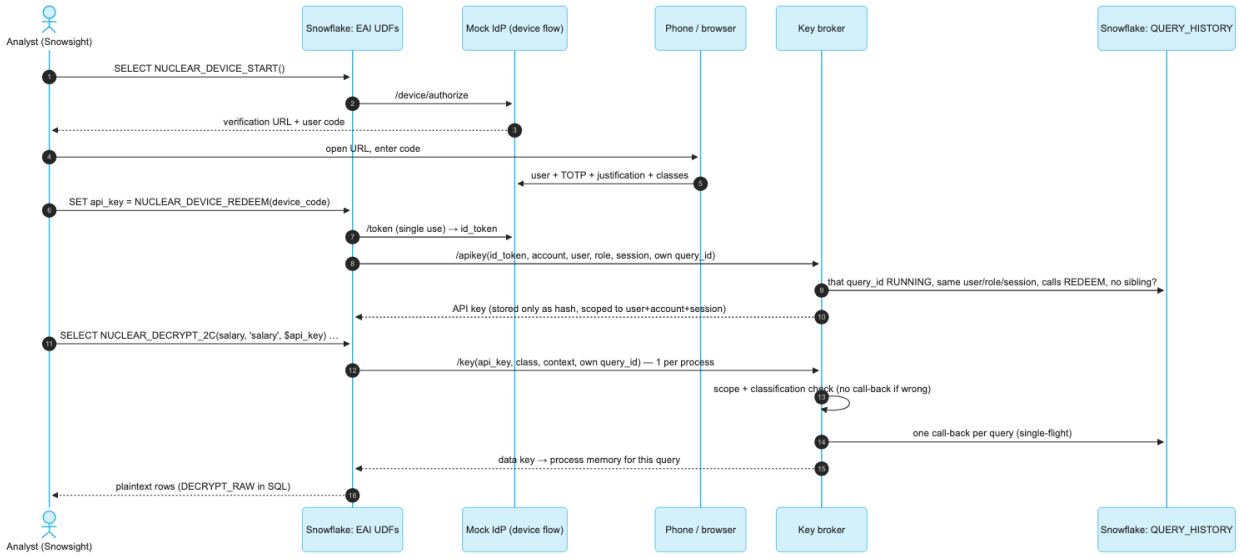


Figure 5 – The IdP issues a single-use device code. The broker exchanges the identity token for a session-scoped API key, and binds each key release to the exact running query.

Analyst flow

```
SELECT item, value FROM TABLE(SOV_DEVICE_LOGIN()) ORDER BY seq;   -- codes, QR, links
-- scan the QR; on the phone enter TOTP and a justification
SET device_code = '<device_code>';
SET api_key = (SELECT SOV_DEVICE_REDEEM($device_code):api_key::STRING);
SELECT department, AVG(TRY_TO_NUMBER(SOV_DECRYPT_2C(salary, 'salary', $api_key)))
FROM EMPLOYEES_ENC GROUP BY 1;
```

The start function returns the approval link as a scannable QR code in the result grid, plus a link to a QR page. On the phone, only the six-digit TOTP has to be typed. The API key value never appears in query text; it lives in a session variable.

Checks at each step

Step	What is checked	Where
Device approval	TOTP of the Snowflake user, entitlement per classification, justification	IdP
Token	Device code single use; identity token valid 5 minutes, audience-restricted	IdP → broker
API key minting	Token signature, subject equals the calling user, token ID single use, **call-back**: the redeeming query is running	Broker
API key	256 random bits, stored only as a SHA-256 hash, scoped to user, account and session and to the approved classifications; TTL up to 8 hours	Broker state
Key fetch	Scope and classification first (no call-back if wrong), then **call-back** on the query's own query ID	Broker

Context values (account, user, role, session and query ID) are added by **secure SQL wrapper functions**, so the analyst passes only the API key. Inside a SQL UDF, `CURRENT_ROLE()` returned the caller's role in testing.

Binding a Key Request to the Running Query

A UDF cannot read its own query ID directly, unlike an external function. Two approaches were implemented and measured:

- **A – self-ID.** The query reads its own ID from `QUERY_HISTORY_BY_SESSION()`, newest first. The pattern is described in a Snowflake community article; here it is wrapped in a secure view. The broker then checks that exact ID: running, same user, role and session, query text calls the protected function, and **no other such query is running in that session**.
- **B – session lookup.** The broker searches for the single running query of that user and session whose text calls the protected function.

TABLE 2 — DOES THE SELF-ID RETURN THE QUERY'S OWN ID? (SNOWFLAKE ONLY, 20 RUNS EACH)

Context	Result
Synchronous query; inside a Python UDF argument; behind a view; behind a secure view	20/20 correct each
Asynchronous single query	20/20
Two asynchronous queries in one session	10/10 own ID (observed, not guaranteed)
Caller's-rights stored procedure, Snowflake Scripting block	Returns the inner statement's ID, which is the statement that calls the UDF
Owner's-rights stored procedure	Query history not accessible; access is denied

TABLE 3 — THROUGH THE BROKER (X-SMALL WAREHOUSE, 15 RUNS EACH)

Scenario	A: self-ID	B: session lookup
Correct binding at 10 / 5,000 / 100,000 rows	45/45	45/45
Back-to-back queries with the same key	**15/15**	8/15 (bound to the previous query)
Forged query ID	Denied	n/a
Finished query ID replayed from outside Snowflake	Denied: `query_not_running(SUCCESS)`	n/a
Two concurrent asynchronous queries in one session	Denied (fails closed)	Denied (ambiguous)
Snowflake call-backs per query	1	1
Added time per query (p50)	about 0.3 s	about 0.6 s

Approach A is the better fit. It is exact, slightly faster, and a forged ID cannot be used.

- **Weakness found and closed:** the first measurement found that two concurrent asynchronous queries in one session could each pass the check with the other's ID. Requiring that no sibling query is running closed this; such cases are now denied.
- **Call-back economy:** call-backs are **single-flighted** per query in the broker. The first request for a query performs the call-back and concurrent requests wait for its decision, so a query costs one call-back regardless of how many processes ask.

End-to-end verification

Check	Result
Redeem before approval	`authorization_pending`
Device code used twice	`invalid_grant`
200 salaries decrypted	200/200, about 3 seconds
Classification not approved	`NULL`
Forged API key	`NULL`
Valid API key from another session of the same user	`NULL`
Stolen key with a finished query ID, sent directly to the broker	403, `query_not_running(SUCCESS)`
Audit	One release line per query, with justification and bound query ID

Considerations

Strengths	Limitations
MFA approval with justification per session, on a second device	IdP and broker to operate (a mock IdP stands in for Okta or Entra ID in the reference)
API key useless outside its session and outside a running query	About 0.3 s added per query
Every key release bound to an exact running query ID and audited	Self-ID pattern is observed behavior, and owner's-rights procedures cannot use it
No API Gateway or API integration; private connectivity possible	API key visible to the same user via `SHOW VARIABLES`
Wrong scope or classification refused without touching Snowflake	Key in compute memory during the query, as in Patterns 2 and 2b

Pattern 2d – Remote Decryption: the Key Never Enters Snowflake

Idea. Pattern 2d keeps the approval and query binding of Pattern 2c, but the broker never returns the data key. A vectorized External Access UDF sends batches of ciphertext (up to 2,000 values) to the broker. The broker checks the same API key scope and query binding, decrypts in the customer's cloud account and returns plaintext. The key is never in Snowflake, not even in query memory. The same shape serves external **de-tokenization**: tokens go out, values come back.

The pattern adapts a decrypt-as-a-service design built for incremental Dynamic Tables, where a pipeline, not a person, is authorised. Pattern 2d adds the per-person, per-query binding of Pattern 2c.

Measured behavior

- 200 salaries decrypted remotely, identical to Pattern 2c; a forged API key returned NULL; the strict variant raised an error when denied.
- Average salary per department over 200 rows: about 4.0 s, against 2.9 s for Pattern 2c and 0.1 s on plaintext. Keeping the key out of Snowflake cost about one second per query at this size.

Considerations

Strengths	Limitations
The data key never enters Snowflake	Plaintext crosses the network on the way back (TLS; private connectivity possible)
Same per-person, per-session approval and query binding as Pattern 2c	Cost grows with rows: one call per batch, decryption in the broker
The broker logs every batch: user, query ID, classification, row count, failures	Same observed-behavior caveat as Pattern 2c
A strict variant raises an error instead of returning NULL, for stored results	Plaintext still exists in Snowflake for the approved query, as in Pattern 2c

Pattern 3 – Split Storage with a Local Join

Idea. The strictest pattern. Snowflake holds **only non-sensitive attributes**. Sensitive attributes stay on-premises. When a combined view is needed, an on-premises notebook pulls the non-sensitive rows from Snowflake, reads the sensitive rows locally and joins them in local memory. Optionally, only k-anonymous aggregates are written back.

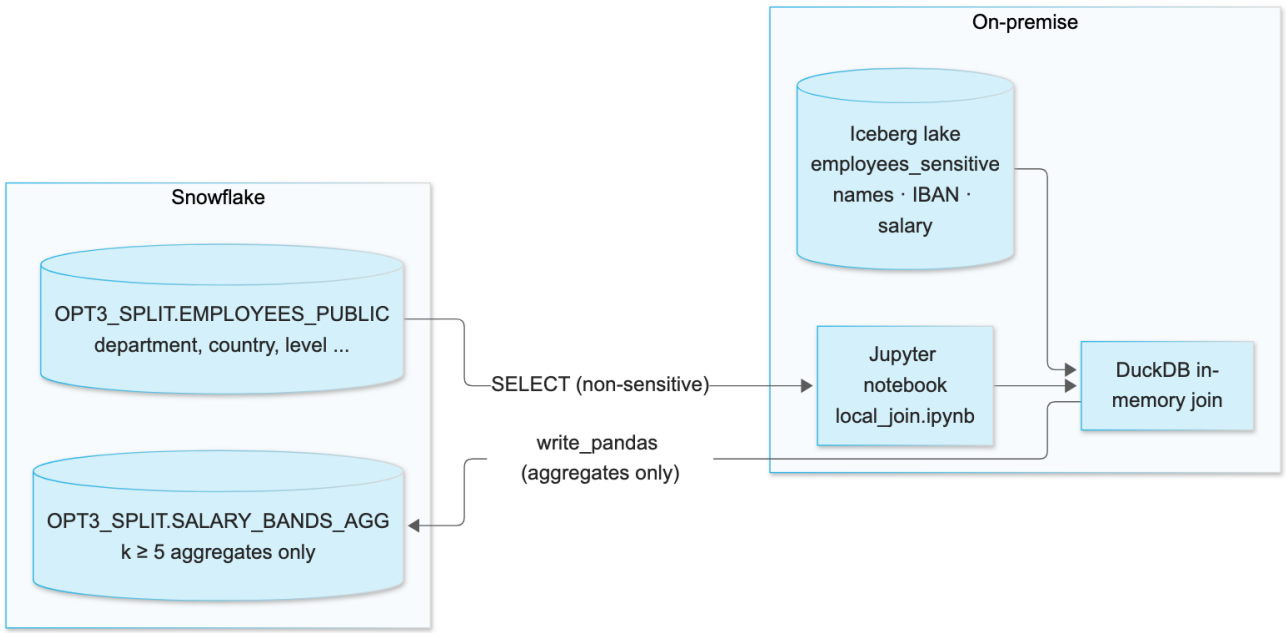


Figure 6 – No tunnel and no inbound connectivity: the on-premises host is an ordinary Snowflake client.

Measured behavior

- 200 non-sensitive rows from Snowflake were joined with 200 sensitive rows from the local Iceberg lake.
- 22 department × country bands with at least five employees were written back. Smaller groups were suppressed.
- `QUERY_HISTORY_BY_SESSION` for the notebook session showed only the read of the public table and the aggregate load. No sensitive value appeared in any Snowflake query, result or stage.

Considerations

Strengths	Limitations
Sensitive data never reaches Snowflake	Joined data is outside Snowflake governance; on-premises governance applies
Lowest operating effort	Compute is limited to the local host
Aggregates can still be shared through Snowflake	Snowflake user experience does not see the combined data
	Aggregates may allow inference; k-anonymity is a simple guard, and differential privacy or aggregation policies could be layered on

Pattern 4 – Ciphertext in Snowflake, Decryption in an On-Premises Application

Idea. Sensitive columns are stored in Snowflake as AES-256 ciphertext, but the key exists **only** in an on-premises analytics application behind OIDC sign-in. The application pulls ciphertext over Arrow, decrypts in memory and caches plaintext in an in-memory DuckDB with a TTL. Snowflake does storage, filtering and aggregation on the plaintext dimensions. It never sees a key, a plaintext value or a plaintext result.

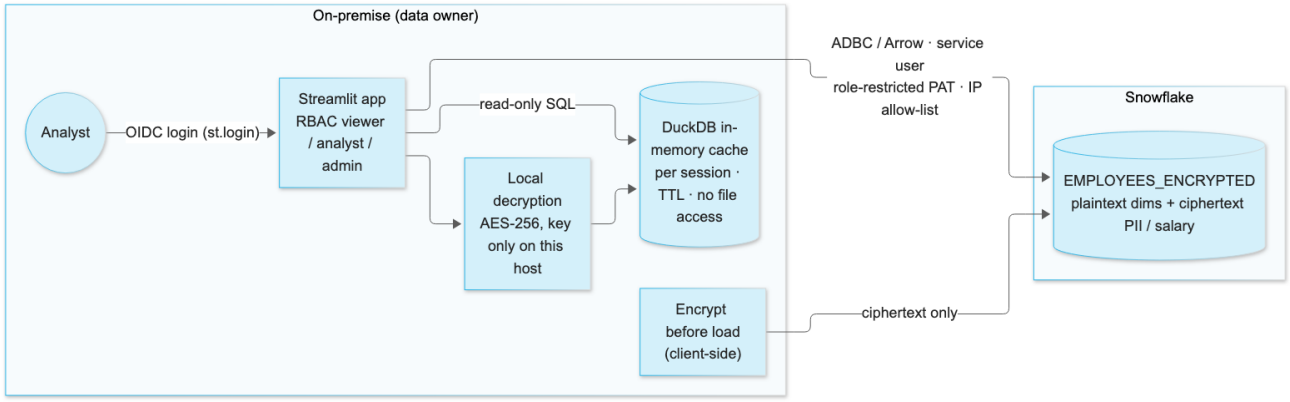


Figure 7 – Encryption happens before load. The application connects as an IP-restricted service user with a role-restricted token.

Application hardening in the reference

- **Query guard:** user SQL is parsed (one read-only statement, no table functions). DuckDB runs with external access disabled and its configuration locked, so attempts like `read_csv('/etc/hosts')` are rejected at both layers.
- **Network:** the server binds to localhost, with CORS enabled.
- **No leakage:** telemetry, browser error details and grid data export are off.
- **Access control:** application roles (admin, analyst, viewer) come from configuration. A development sign-in exists only when explicitly enabled, and shows a warning banner when it is.

Measured behavior

- 200 rows decrypted with 0 failures. Local per-department averages matched the plaintext source.
- The same row shows ciphertext in live source mode and plaintext in local cache mode.
- Write statements, multi-statement SQL and file-reading table functions are rejected.

Considerations

Strengths	Limitations
No key ever enters Snowflake, not even transiently	Snowflake user experience (Snowsight, Cortex, BI) sees only ciphertext
Storage, backup and sharing stay in Snowflake	Aggregations on encrypted columns cannot be pushed down
One application to operate	Key on the application host; an HSM or KMS lookup could replace a secrets file
	The reference uses AES-CBC; AES-GCM, as in Pattern 2, would also detect tampering

Complement – Homomorphic Encryption for Numeric Columns

Idea. Some numeric columns, such as salaries, bonuses and rates, never need to be seen in Snowflake, only computed on. With homomorphic encryption (Microsoft SEAL through Pyfhel, in Python UDFs), Snowflake computes on ciphertexts. Only the data owner can decrypt the result. Snowflake holds only the public context; no secret key exists in Snowflake, not even transiently.

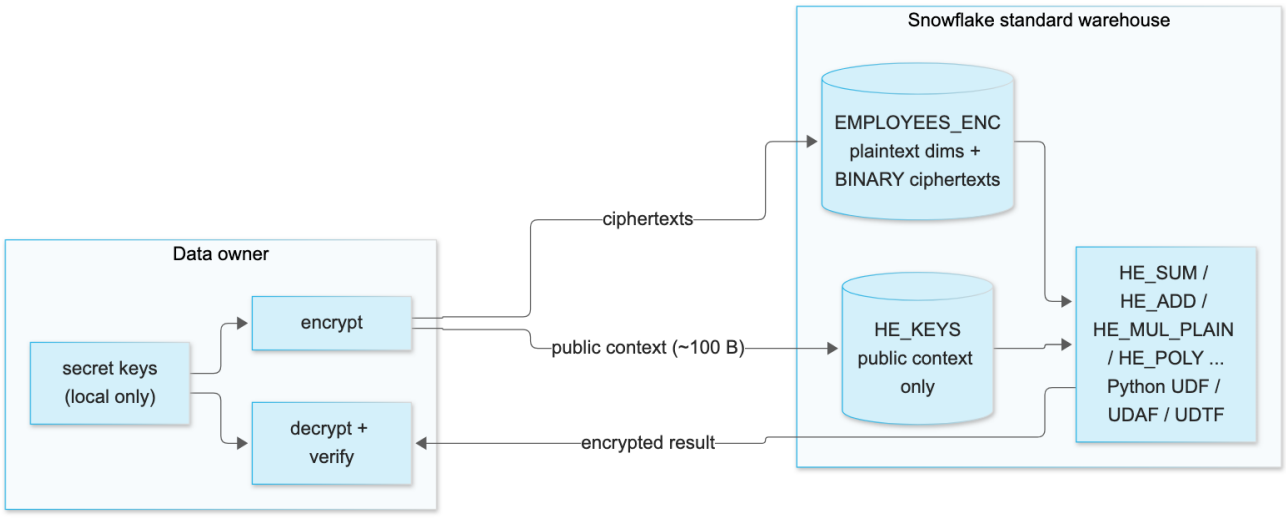


Figure 8 – Ciphertexts and the public context are stored in Snowflake. Results are decrypted and verified by the data owner.

Works on ciphertext	Not possible with these schemes
`SUM`, `COUNT`, `AVG`; `GROUP BY` on plaintext dimensions	`WHERE salary > x`, `ORDER BY`, `MIN`/ `MAX`
Net pay = salary + bonus − deductions	Joins on encrypted keys
Constant arithmetic (raises, currency conversion)	Division by an encrypted value
Variance, standard deviation, correlation, regression (CKKS)	
Linear and polynomial model scoring	

In the reference environment, 13 homomorphic functions were deployed and 100 encrypted employees loaded. Three notebooks executed with 0 errors, and every decrypted result was checked against plaintext ground truth.

Ciphertexts are large: about 86 KB per value for BFV and 300 KB for CKKS; packed (SIMD) mode puts a whole column into one ciphertext. This suits targeted, high-value columns rather than whole tables. Homomorphic encryption combines well with Pattern 2 (identifying columns readable under step-up, numeric columns never decrypted) and Pattern 3 (aggregates computed in Snowflake, decrypted on-premises).

Comparing and Combining the Patterns

Table 4 — The patterns at a glance

	Where the sensitive data lives	Who holds the switch	When Snowflake can see plaintext	Snowflake user experience
P1 Tunnel	On-premises Iceberg lake	Data owner: tunnel on/off, TTL	During a query while the tunnel is open	Yes
P2 Key broker	Snowflake, AES-256-GCM ciphertext	Data owner: KMS and broker	During a running query of a stepped-up user	Yes, via masking policy
P2b EAI key fetch	Snowflake, AES-256-GCM ciphertext	Data owner: vault authorizes the integration	During a query (key memo per query)	Yes, via policy or view
P2c Device flow	Snowflake, AES-256-GCM ciphertext	Data owner: IdP approval and broker	During a running query in the approved session	Yes, via SQL function
P2d Remote decrypt	Snowflake, AES-256-GCM ciphertext; key only in the customer's cloud	Data owner: IdP approval and broker	Plaintext (never the key) during a running query in the approved session	Yes, via SQL function
P3 Split storage	On-premises only	Data owner; data never leaves	Never	No (local notebook)
P4 Client decrypt	Snowflake, AES-256 ciphertext	Data owner: key only in on-premises app	Never	No (on-premises app)
HE	Snowflake, homomorphic ciphertext	Data owner: secret key stays local	Never	Partly (encrypted aggregates)

Table 5 — Operational trade-offs

	Added latency per query (reference)	Operating effort	Typical fit
P1	Network-bound	SPCS service, agent, lake	Data that may not be stored in a foreign cloud but must be queryable with the Snowflake UX
P2	About 4 s	AWS stack and API integration	Highly sensitive columns under external key control with full UX
P2b	One key fetch per process	Minimal	"Key stays with us" with little infrastructure
P2c	About 0.3 s	IdP and broker	Per-person, per-session approval without API Gateway
P2d	About 1 s more than P2c (one broker call per batch)	IdP and broker (as P2c)	"The key must never be in the platform" with the Snowflake UX kept
P3	Local host	Lowest	Data that must never leave the premises
P4	Pull and decrypt locally	One application	A small, trusted on-premises analyst group
HE	Heavy	Python UDFs, key handling	Numbers that only need arithmetic

Relation to the EU Cloud Sovereignty Framework (SEAL)

The European Commission's Cloud Sovereignty Framework (version 1.2.1, October 2025) assesses cloud services against eight Sovereignty Objectives, SOV-1 to SOV-8. Each objective is rated on a Sovereignty Effectiveness Assurance Level (SEAL):

- **SEAL-0:** no sovereignty;
- **SEAL-1:** jurisdictional sovereignty;
- **SEAL-2:** data sovereignty;
- **SEAL-3:** digital resilience;
- **SEAL-4:** full digital sovereignty.

A procurement sets a minimum level per objective, and official levels result only from such an assessment.

The patterns in this paper act mainly on **SOV-3 (Data and AI Sovereignty)**, and only for the protected dataset. They do not change how the Snowflake service itself is rated on strategic, legal, operational or supply-chain objectives (SOV-1, SOV-2, SOV-4 and SOV-5). The table below is the author's indicative reading, not an assessment or a certified rating.

TABLE 6 — INDICATIVE SOV-3 LEVEL FOR THE PROTECTED DATA

	Indicative level	Reasoning
Baseline (native controls)	SEAL-1	The provider holds both the data and the keys.
P2b EAI key fetch	SEAL-1 to SEAL-2	The customer holds the key, but the vault cannot tell which person or query is asking.
P2 Key broker	SEAL-2	Each query is approved, but plaintext exists in Snowflake while the query runs.
P2c Device flow	SEAL-2	As P2, with approvals bound to a person and a session.
P2d Remote decrypt	SEAL-2	As P2c, and the key is never in Snowflake; plaintext still exists there while the approved query runs.
P1 Tunnel	SEAL-2 while open, SEAL-3 while closed	The data stays on premises, but Snowflake processes plaintext while the tunnel is open.
P4 Client decrypt	SEAL-3	Snowflake holds only ciphertext, and the key never leaves the customer.
HE	SEAL-3	Snowflake computes on ciphertext, and only the customer can read results.
P3 Split storage	SEAL-3 to SEAL-4	The sensitive data never reaches Snowflake; the level depends on the customer's own stack.

One possible layering

The patterns are not mutually exclusive. One possible estate:

- native Snowflake data, with Tri-Secret Secure and masking, for the bulk of the data;
- Pattern 2 or 2c for identifying columns;
- homomorphic encryption for salary figures;
- Pattern 1 or 3 for the few datasets that may not be stored in the cloud at all.

Security Caveats and Operational Considerations

Identity Pass-Through to On-Premises Sources Is a Customer Design Task

This is a major gap that each customer has to design for. The reference implementation does not pass the end user's identity through to on-premises data sources. In Pattern 1 and its notebook variant, the on-premises agent authenticates to the Iceberg catalog with its own credentials. Who may open the tunnel is controlled by the data owner, and who may call the function or open the notebook is controlled by Snowflake RBAC. The on-premises source itself, however, does not see or verify the individual Snowflake user.

Seamless, per-user authentication from Snowflake (for example from a notebook on Snowpark Container Services) to an on-premises source has to be designed by the customer. **Options**:

- access control handled out of band, on the agent or the catalog side;
- a token from the customer's identity provider, for example through an OAuth device-code flow started from the notebook with helper functions or UDFs, similar to Pattern 2c;
- an engineered proof of the Snowflake identity that the on-premises side can verify, for example derived from the user context, request headers in Snowpark Container Services, or secrets made available to the notebook.

The reference implementation leaves this out deliberately. On-premises sources reachable through the tunnel, and their authentication requirements, vary too widely for one pattern to fit. An example pattern for seamless Iceberg catalog authentication from a notebook on Snowpark Container Services is planned.

Plaintext Exists During Authorized Processing

In Patterns 1, 2, 2b, 2c and 2d, plaintext values exist in Snowflake compute memory while an authorized query runs; in Patterns 2–2c the data key does too, in Pattern 2d it does not. These patterns govern when and for whom decryption happens. **Options**: Patterns 3, 4 or homomorphic encryption keep processing out of Snowflake or on ciphertext.

Key Custody Is Not Data Custody

"The key stays with us" and "the data stays with us" are different requirements. Decrypting upstream, before the data is loaded, also keeps the key out of Snowflake, but it puts plaintext in Snowflake from the first table onwards. Patterns 2 to 2d keep the key with the customer; Patterns 1, 3 and 4 keep the readable data with the customer. A requirement should state which of the two it means.

Failing Closed When Results Are Stored

The decrypt functions return NULL (or a placeholder such as `**KEY DENIED**`) when access is denied. That suits interactive queries. Where the result is stored, through `CREATE TABLE AS` or an incremental Dynamic Table, a denial, an unreachable broker or an unknown key would be committed as a value, and an incremental Dynamic Table never revisits committed rows. **Options**: a strict variant that raises an error instead (Pattern 2d's `SOV_DECRYPT_2D_STRICT`), so the statement or refresh fails and is retried.

Dynamic Tables

External functions, used by Pattern 2's masking policies, are not supported in Dynamic Tables. External Access UDFs (Patterns 2b, 2c, 2d) are; they should be `VOLATILE`, because replacing an `IMMUTABLE` UDF re-initializes the Dynamic Table and decrypts all history again. Refreshes run as a system user, so per-person binding does not apply: a pipeline variant authorises the pipeline, not a person.

Key Rotation

The reference holds one data key per classification, so rotating a key means re-encrypting the data. A key identifier in each envelope lets old and new keys coexist; the broker then reloads its key set when it sees an unknown identifier. Without that reload, values encrypted under a new key fail to decrypt, which is where failing closed matters.

Authorized Users Can Copy What They See

A user who may read plaintext can persist it (`CREATE TABLE AS`, export, screenshots). **Options**: no `CREATE TABLE` privilege for decrypting roles, `PREVENT_UNLOAD_TO_INLINE_URL` and related parameters, restricted stages, short approval windows, and audit of every key release by query ID.

The Result Cache and `RESULT_SCAN`

Queries containing external functions are not served from the persisted result cache. Pattern 1 uses `VOLATILE` and a nonce for the same effect, and the reference also sets `USE_CACHED_RESULT = FALSE` on users. `RESULT_SCAN(<query_id>)` by the same user within 24 hours could still show a past result. That is worth stating explicitly in a risk assessment.

Observed Behavior Is Not a Contract

The per-query UDF memo (Pattern 2b) and the self-ID pattern (Pattern 2c) are observed behaviors. **Options**: design so that a change fails closed rather than open. Examples: memo TTLs keyed by user and role, the sibling-query check, re-running the bake-off (`pixi run bench`) after platform updates, and keeping the session-lookup mode as a fallback.

Call-Back Privileges and Placement

Patterns 2 and 2c need a service user that can read other users' query history. Granting only `MONITOR` on the relevant warehouse kept this least-privileged in testing, and it limited decryption to monitored warehouses. The service user's network policy depends on where the broker runs. Serverless functions have dynamic egress IPs unless routed through a fixed NAT address.

Key Caching in the Broker

The brokers keep the unwrapped data key in memory so that KMS is not called for every request. In the reference the cache entry is renewed every 30 minutes (`KEY_CACHE_TTL_S`). The interval is a trade-off: a shorter one makes a revoked KMS grant or a disabled key effective sooner, at the cost of more KMS calls; a longer one keeps plaintext key material in memory longer. Each Lambda instance keeps its own cache, so revocation can take up to one interval everywhere. Approval checks (grants, API keys, query binding) are not cached this way and apply to every query.

Owner's-Rights Procedures

Inside an owner's-rights stored procedure, the query history of the current user is not accessible, so Pattern 2c denies key release there. **Options**: caller's-rights procedures, or Pattern 2 (external function headers), for procedural workloads.

Secrets in Session Variables

In Pattern 2c, the API key sits in a session variable. It never appears in query text or history, but `SHOW VARIABLES` displays it to the same user in that session. The key's scope (user, account, session) and TTL limit the value of a copy.

Identity Provider and MFA Choice

The reference uses TOTP and a mock IdP. **Options**: the corporate IdP's device flow or step-up (`acr` claims), push-based MFA, and four-eyes approval for the most sensitive classifications.

Reference Implementation

All patterns are implemented, deployed and verified in a demo account (AWS eu-central-1). They use a shared synthetic dataset of 200 employees: public columns are department, country and level; sensitive columns are name, national ID, IBAN, salary and bonus.

Folder	Contents
`opt1-tunnel/`	SPCS sidecar and query agent, on-premises agent, `tunnel_ctl.py` with TTL and audit, Lakekeeper and an S3-compatible store
`opt2-keybroker/`	Encrypt-and-load client, TOTP enrollment, broker and step-up Lambdas, CDK stack, external function and masking policies
`opt2b-warm-cache/`	Probe UDFs measuring memo lifetime
`opt2c-device-flow/`	Mock IdP (device flow, QR), broker for External Access UDFs (key release and remote decryption, Pattern 2d), SQL wrappers, bake-off and end-to-end tests
`opt3-local-join/`	Local-join notebook and aggregate write-back
`opt4-client-decrypt/`	Streamlit application with OIDC, query guard and decryption pipeline
`bonus-homomorphic/`	Homomorphic UDFs (SEAL via Pyfhel) and notebooks
`snowsight_demo.sql`	Walkthrough of every pattern in Snowsight
`docs/`	Per-pattern notes, diagrams and a slide deck

Each folder has a `pixi.toml` with tasks for setup, tests and demos. The reference implementation is a demonstration, not a supported product. A production deployment would replace the mock IdP, design identity pass-through to on-premises sources (see Identity Pass-Through to On-Premises Sources Is a Customer Design Task), review key custody and rotation, and run its own threat assessment.

Conclusion

Snowflake's native controls cover most data well. For the small set of datasets where an organization requires its own participation before anyone can read them, customer-held controls can be layered on top. These controls can take three forms:

- **the data itself** stays with the customer (Patterns 1 and 3);
- **the key** stays with the customer and is released per approved query (Patterns 2, 2b, 2c), or never leaves the customer while values are decrypted for an approved query (Pattern 2d);

- **the key never leaves** the customer, and Snowflake computes only on ciphertext or not at all (Pattern 4, homomorphic encryption).

Each pattern trades user experience, latency and operating effort for control in a different way. The measurements in this paper are intended to make those trades concrete:

- one call-back per query;
- about 0.3 seconds added for query-bound key release in Pattern 2c;
- no reuse of results across queries;
- no sensitive value in Snowflake's history for split storage.

One design task remains with the customer in every deployment: passing the end user's identity through to on-premises sources. The reference implementation does not solve it, because the right approach depends on the source and the organization's identity infrastructure.

Table 7 — Capabilities used

Capability	Status
Snowpark Container Services (service functions, outbound connectivity)	Generally available
External functions with `CONTEXT_HEADERS`	Generally available
External Access Integrations, network rules, secrets	Generally available
Dynamic data masking, secure views and functions	Generally available
`DECRYPT_RAW` / `TRY_DECRYPT_RAW` (AES-GCM)	Generally available
`INFORMATION_SCHEMA.QUERY_HISTORY_BY_SESSION` / `_BY_USER`	Generally available
Python UDFs and table functions (Anaconda packages, including `qrcode`)	Generally available
Tri-Secret Secure	Generally available (Business Critical)
External tokenization	Generally available (Enterprise and above)
Per-query UDF memo; self query ID from session history	Observed behavior, not documented